# Computation Vision - Project 2 Report

**Topic A3: People and vehicle detection using thermal imagery**

Authors:

- David Marafuz Gaspar - 106541 - david.marafuz.gaspar@tecnico.ulisboa.pt
- Pedro Gaspar Monico - 106626 - pedro.monico@tecnico.ulisboa.pt

Supervisors:

- Prof. Alexandra Moutinho
- Eng. Alexandre Athayde

**Table of Contents**

1. Introduction....1
2. Dataset Analysis and Preparation....2
  2.1 Dataset Overview....2
  2.2 Category Analysis....5
  2.3 Target Class Selection....6
  2.4 Data Preparation....7
3. Data Storage and Augmentation Strategy....9
  3.1. Data Storage....9
  3.2. Data Augmentation Strategy....11
4. Model Architecture Pipeline....12
  4.1. Baseline Selection Strategy....12
    4.1.1. Comparative Architecture Overview....13
  4.2. Transfer Learning Implementation....13
  4.3. Training and Monitoring....14
  4.4. Model Comparison and Selection....14
    4.4.1. Evaluation Metrics....14
    4.4.2. Model Selection....15
5. Model Analysis and Comparison....15
  5.1. High-Speed Baselines....15

## 1. Introduction

With the rapid advancement of autonomous driving and Advanced Driver-Assistance Systems (ADAS), the ability to reliably detect pedestrians and vehicles in real-time has become a fundamental requirement for safe road navigation. Traditional RGB-based vision systems, while effective under standard conditions, are inherently limited by their dependence on visible light, making them prone to failure in challenging environments such as darkness, fog, heavy rain, or glare which are precisely the conditions where robust perception is most critical for the safety of both passengers and other road users.

Thermal imaging offers a compelling alternative, as it captures heat signatures emitted naturally by humans and vehicles rather than relying on reflected light. This makes thermal-based detection largely invariant to lighting conditions, providing a more consistent and reliable perception pipeline for safety-critical applications. Unlike RGB cameras, thermal sensors detect temperature differences between objects and their surroundings, preserving the distinct silhouettes of pedestrians and vehicles even in the most adverse scenarios. These properties make thermal sensors an increasingly attractive complement to visible-light cameras in modern ADAS systems.

As part of Topic A3 of the Intelligent Vision Systems course, this project aims to develop and optimize a deep learning-based object detection model using the FLIR ADAS dataset. The goal is to build a system capable of accurately detecting pedestrians and vehicles in real driving scenarios. Specifically, this report details the following steps:

- **Dataset Analysis and Preparation:** Characterization of the FLIR ADAS thermal dataset, identification of relevant object classes, and the structuring of data for training, validation, and testing;
- **Baseline Model:** The design and deployment of an initial object detection pipeline using simple Deep Learning Architecture from what we will improve on;
- **Model Evaluation:** Rigorous training and assessment of the baseline model using standard computer vision metrics to analyze its initial detection capabilities;
- **System Optimization:** Iterative improvement of the detection system through the exploration of advanced architectures, hyperparameter tuning, targeted preprocessing, and data augmentation strategies.

This project explores the development and optimization of a deep learning-based object detection system, demonstrating how careful dataset preparation, model design, and iterative improvement can enhance detection performance, ultimately contributing to safer autonomous navigation.

**Attention**: In order to reduce clutter in the report, auxiliary scripts (specifically those implemented for subsequent processing steps and for plotting purposes) have been organized into separate files and are only referenced in the code when invoked. Auxiliary functions follow the naming convention `aux_<function>.m`, while plotting functions follow `plot_<function>.m`.

# 2. Dataset Analysis and Preparation

## 2.1 Dataset Overview

Before designing and training the object detection architecture, it is essential to understand the underlying data distribution. The FLIR ADAS dataset provides annotations in the standard MS-COCO JSON format [1] and is pre-divided into three dedicated splits: Training, Validation, and Testing. Each split is stored in a separate folder and contains a `coco.json` file with the corresponding image metadata and bounding box annotations. The

script below loads each split, parses the COCO annotations, and computes the number of images per split along with their relative proportions. The resulting distribution is summarised in Table 1.

[1] TELEDYNE | FLIR OEM Documentation: https://oem.flir.com/en-150/solutions/automotive/adas-dataset-form/

```
%% Define Dataset Paths and Splits
datasetPaths = {'./Data/images_thermal_train', './Data/images_thermal_val',
'./Data/video_thermal_test'};
splitNames = {'Train', 'Validation', 'Test'};

%% Read Categories from the Training Dataset
cocoPath = fullfile(datasetPaths{1}, 'coco.json');
cocoData = jsondecode(fileread(cocoPath));

numClasses = length(cocoData.categories);
classNames = strings(1, numClasses);
classIDs = zeros(1, numClasses);

for i = 1:numClasses
    classNames(i) = cocoData.categories(i).name;
    classIDs(i) = cocoData.categories(i).id;
end

numImagesPerSplit = zeros(1, length(datasetPaths));
countsPerSplit = zeros(numClasses, length(datasetPaths));

%% Process Each Dataset Split
for d = 1:length(datasetPaths)
    cocoPath = fullfile(datasetPaths{d}, 'coco.json');
    cocoData = jsondecode(fileread(cocoPath));

    numImagesPerSplit(d) = length(cocoData.images);

    if d == 1
        numImgs   = length(cocoData.images);
        filenames   = cell(numImgs, 1);
        peopleBoxes  = cell(numImgs, 1);
        vehicleBoxes = cell(numImgs, 1);

        imageIDList = zeros(numImgs, 1);
        for k = 1:numImgs
            filenames{k}   = fullfile(datasetPaths{1},
cocoData.images(k).file_name);
            imageIDList(k) = cocoData.images(k).id;
        end
    end

    lowerClassNames = lower(classNames);
```

```
    targetClass1 = "person";
    targetClass2 = ["car", "motorcycle", "bus", "train", "truck", "other
vehicle"];

    annotations = cocoData.annotations;
    for i = 1:length(annotations)
        if iscell(annotations)
            ann = annotations{i};
        else
            ann = annotations(i);
        end

        idx = find(classIDs == ann.category_id);
        if ~isempty(idx)
            countsPerSplit(idx, d) = countsPerSplit(idx, d) + 1;
        end

        % Store bounding boxes for train split visualization
        if d == 1
            catName = lower(classNames(idx));
            imgIdx  = find(imageIDList == ann.image_id, 1);
            bbox    = ann.bbox; % [x, y, w, h]

            if catName == targetClass1
                peopleBoxes{imgIdx} = [peopleBoxes{imgIdx}; bbox];
            elseif any(catName == targetClass2)
                vehicleBoxes{imgIdx} = [vehicleBoxes{imgIdx}; bbox];
            end
        end
    end
end

% Number and Percentage of Images per Split
totalImages = sum(numImagesPerSplit);

pcts = arrayfun(@(x) sprintf('%.2f%%', (x/totalImages)*100),
numImagesPerSplit, 'UniformOutput', false);

T_splits = table(splitNames', numImagesPerSplit', pcts', ...
    'VariableNames', {'Split', 'N_Images', 'Percentage'});

disp('Table 1: Image Count and Percentage by Dataset Split');
```

```
Table 1: Image Count and Percentage by Dataset Split
```

```
disp(T_splits);
```

| Split | N_Images | Percentage |
|---|---|---|
| {'Train'} | 10742 | {'68.70%'} |

```
{'Validation'}      1144      {'7.32%' }
{'Test'      }      3749      {'23.98%'}
```

Based on the script's output, the data distribution is:

- Training Set: 10,742 images (68.70%)
- Validation Set: 1,144 images (7.32%)
- Testing Set: 3,749 images (23.98%)

This distribution follows standard deep learning practice, allocating the majority of the data to training while reserving a substantial portion for final evaluation. The test set, comprising approximately 24% of the total data, provides a sufficiently large and unseen sample to produce reliable performance estimates. The validation set, although smaller, is used exclusively during training to monitor generalisation and guide hyperparameter tuning.

## 2.2 Category Analysis

The FLIR ADAS dataset originally contains a large number of annotated categories, however many of these have zero instances across all splits and are therefore disregarded. Table 2 presents only the 16 categories that have at least one annotated instance in the dataset.

```
% Full Category Table (all classes in dataset)
validIdx = any(countsPerSplit > 0, 2);  % categories with at least 1 instance

T_categories = table(...
    classNames(validIdx)', ...
    classIDs(validIdx)', ...
    countsPerSplit(validIdx, 1), ...
    countsPerSplit(validIdx, 2), ...
    countsPerSplit(validIdx, 3), ...
    'VariableNames', {'Category', 'ID', 'Train', 'Val', 'Test'});

disp('Table 2: All Categories with Non-Zero Instance Counts');
```

```
Table 2: All Categories with Non-Zero Instance Counts
```

```
disp(T_categories);
```

| Category | ID | Train | Val | Test |
|---|---|---|---|---|
| "person" | 1 | 50478 | 4470 | 12323 |
| "bike" | 2 | 7237 | 170 | 113 |
| "car" | 3 | 73623 | 7133 | 30517 |
| "motor" | 4 | 1116 | 55 | 3314 |
| "bus" | 6 | 2245 | 179 | 0 |
| "train" | 7 | 5 | 0 | 0 |

```
"truck"             8      829      46     2634
"light"            10    16198    2005     6758
"hydrant"          11     1095      94      277
"sign"             12    20770    2472     5660
"dog"              17        4       0       25
"deer"             18        8       0        0
"skateboard"       37       29       3        0
"stroller"         73       15       6        0
"scooter"          75       15       0        0
"other vehicle"    79     1373      63      696
```

As shown in Table 2, the dataset includes 16 active categories: Person, Bike, Car, Motor, Bus, Train, Truck, Light, Hydrant, Sign, Dog, Deer, Skateboard, Stroller, Scooter, and Other Vehicle. A clear class imbalance is visible since the classes Person and Car dominate across all splits, while categories such as Train, Dog, Deer, Skateboard, Stroller, and Scooter have very few instances, particularly in the validation and test sets.

In line with the objectives of Topic A3, which focuses on detecting pedestrians and vehicles in real driving scenarios, the dataset is filtered to retain only the 7 most relevant categories: Person, Car, Motorcycle, Bus, Train, Truck, and Other Vehicle. Categories unrelated to road safety such as Traffic Light, Fire Hydrant, Street Sign, and animals are excluded from the detection pipeline.

## 2.3 Target Class Selection

As evidenced by Table 2, the dataset exhibits a significant class imbalance. Person and Car account for the vast majority of annotated instances, while categories such as Train, Motorcycle, and Other Vehicle are considerably underrepresented. Training a multi-class detector on this raw distribution would likely result in a model biased towards the dominant classes, with poor generalisation on the minority ones.

To address this, the detection task is reformulated into two well-balanced and semantically meaningful classes:

- Class 1 **(People):** directly mapped from the Person category, representing all pedestrians in the scene.
- Class 2 **(Vehicles):** formed by merging Car, Motorcycle, Bus, Train, Truck, and Other Vehicle into a single class. This grouping is justified by the shared detection objective for identifying any road vehicle ensuring that the combined class has sufficient training examples for reliable detection.

The instance counts for the selected categories are presented in Table 3. This grouping strategy ensures the model focuses on the objects most critical to road safety, while mitigating the effects of class imbalance through category consolidation.

```
% Define target categories and display instance counts
targetCategories = ["person", "car", "motor", "bus", "train", "truck",
"other vehicle"];

catLabels  = strings(length(targetCategories), 1);
trainCounts = zeros(length(targetCategories), 1);
valCounts   = zeros(length(targetCategories), 1);
testCounts  = zeros(length(targetCategories), 1);
```

```
for c = 1:length(targetCategories)
    idxCat = find(lowerClassNames == targetCategories(c));
    if ~isempty(idxCat)
        catLabels(c)   = classNames(idxCat);
        trainCounts(c) = countsPerSplit(idxCat, 1);
        valCounts(c)   = countsPerSplit(idxCat, 2);
        testCounts(c)  = countsPerSplit(idxCat, 3);
    end
end

T_target = table(catLabels, trainCounts, valCounts, testCounts, ...
    'VariableNames', {'Category', 'Train', 'Val', 'Test'});

disp('Table 3: Instance Counts for Target Categories');
```

```
Table 3: Instance Counts for Target Categories
```

```
disp(T_target);
```

| Category | Train | Val | Test |
|---|---|---|---|
| "person" | 50478 | 4470 | 12323 |
| "car" | 73623 | 7133 | 30517 |
| "motor" | 1116 | 55 | 3314 |
| "bus" | 2245 | 179 | 0 |
| "train" | 5 | 0 | 0 |
| "truck" | 829 | 46 | 2634 |
| "other vehicle" | 1373 | 63 | 696 |

## 2.4 Data Preparation

While the full FLIR ADAS dataset provides over 14,000 training images, training on the complete dataset imposes significant computational demands in terms of memory, processing time, and GPU requirements. Given the hardware constraints typical of an academic setting, and in the interest of rapid iteration during model development and hyperparameter tuning, a representative 10% subset was sampled from each split.

This decision is supported by several practical arguments. First, the dataset is heavily dominated by two classes (Person and Vehicle), meaning that even a 10% sample retains a statistically representative distribution of instances across classes. Second, reducing the dataset size allows for faster training cycles, making it feasible to experiment with different architectures, augmentation strategies, and training parameters within a reasonable timeframe. Third, it is standard practice in deep learning research to validate a pipeline on a smaller subset before scaling up, as it helps identify implementation errors and convergence issues early without incurring the full computational cost.

The subsampling is performed by the auxiliary function `aux_subsampleDataset.m`, which randomly samples a specified fraction of images from each split while preserving the original folder structure and COCO annotation format. The resulting subsets are saved to new folders suffixed with `_subset`.

```
subsetFraction = 0.10;   % 10% of each split
rngSeed        = 42;     % for reproducibility

splitPaths = {
    './Data/images_thermal_train',
    './Data/images_thermal_val',
    './Data/video_thermal_test'
};

subsetPaths = cell(size(splitPaths));
for d = 1:numel(splitPaths)
    subsetPaths{d} = aux_subsampleDataset(splitPaths{d}, subsetFraction,
rngSeed);
    fprintf('Subset created: %s\n', subsetPaths{d});
end
```

```
   Copied 1074 / 1074 images
   [./Data/images_thermal_train] 10742 → 1074 images (10%) saved to ./Data/images_thermal_train_subset
Subset created: ./Data/images_thermal_train_subset
   Copied 114 / 114 images
   [./Data/images_thermal_val] 1144 → 114 images (10%) saved to ./Data/images_thermal_val_subset
Subset created: ./Data/images_thermal_val_subset
   Copied 375 / 375 images
   [./Data/video_thermal_test] 3749 → 375 images (10%) saved to ./Data/video_thermal_test_subset
Subset created: ./Data/video_thermal_test_subset
```

```
% Stats table
splitLabels  = {'Train'; 'Validation'; 'Test'};
fullCounts   = numImagesPerSplit';
subsetCounts = zeros(3,1);
for d = 1:3
    subCocoPath = fullfile(subsetPaths{d}, 'coco.json');
    subCoco     = jsondecode(fileread(subCocoPath));
    subsetCounts(d) = numel(subCoco.images);
end
retained = arrayfun(@(a,b) sprintf('%.1f%%', 100*a/b), ...
                    subsetCounts, fullCounts, 'UniformOutput', false);

T_subset = table(splitLabels, fullCounts, subsetCounts, retained, ...
    'VariableNames', {'Split','Full_Dataset','Subset','Retained'});

disp('Table 4: Full Dataset vs. Subset Size per Split');
```

```
Table 4: Full Dataset vs. Subset Size per Split
```

```
disp(T_subset);
```

| Split | Full_Dataset | Subset | Retained |
|---|---|---|---|
| {'Train'} | 10742 | 1074 | {'10.0%'} |

```
    {'Validation'}        1144          114     {'10.0%'}
    {'Test'      }        3749          375     {'10.0%'}
Grouped Instance Counts
Class                | Train   | Val     | Test
-------------------------------------------------------
Class 1 (People)     | 50478   | 4470    | 12323
Class 2 (Vehicles)   | 78075   | 7421    | 33847
```

# 3. Data Storage and Augmentation Strategy

## 3.1. Data Storage

To train a deep learning model in MATLAB, the dataset must be efficiently organized and loaded using datastores. This approach enables scalable data handling and ensures that images and their corresponding annotations are correctly synchronized during training.

For the object detection task, two primary datastores are used:

- *imageDatastore*: Responsible for loading and managing the thermal image files from the dataset. It provides efficient access to image data during training without requiring all images to be stored in memory simultaneously.
- *boxLabelDatastore*: Stores the ground-truth annotations, including bounding box coordinates and their associated class labels (People and Vehicles). This datastore ensures that each image is paired with the correct object annotations.

These two datastores are combined into a single unified datastore, allowing the model to receive both the input image and its corresponding labels at each training iteration. This synchronization is essential for supervised learning, where the model learns to associate image features with the correct object locations and classes.

```
%% Data Loading (Creating Ground Truth Tables and Datastores)
```

```
% Identify which Category IDs belong to the two target classes
class1_IDs = classIDs(lowerClassNames == targetClass1);
class2_IDs = classIDs(ismember(lowerClassNames, targetClass2));

combinedDatastores = cell(1, length(datasetPaths));

% Loop through Train, Val, and Test splits
for d = 1:length(datasetPaths)

    % Read JSON for the current split
    cocoPath = fullfile(datasetPaths{d}, 'coco.json');
    cocoData = jsondecode(fileread(cocoPath));

    numImgs = length(cocoData.images);
    filenames = cell(numImgs, 1);
    peopleBoxes = cell(numImgs, 1);
    vehicleBoxes = cell(numImgs, 1);

    % Create a map (dictionary) to link image_id to the array index
    imgID_to_idx = containers.Map('KeyType', 'int32', 'ValueType', 'int32');

    % Extract image filenames and initialize empty bounding box arrays
    for i = 1:numImgs
        img = cocoData.images(i);
        if iscell(img), img = img{1}; end

        filenames{i} = fullfile(datasetPaths{d}, img.file_name);
        imgID_to_idx(int32(img.id)) = int32(i);

        % Initialize empty bounding box arrays [x, y, width, height]
        peopleBoxes{i} = zeros(0, 4);
        vehicleBoxes{i} = zeros(0, 4);
    end

    % Extract and filter Bounding Boxes from annotations
    annotations = cocoData.annotations;
    for i = 1:length(annotations)
        ann = annotations(i);
        if iscell(ann), ann = ann{1}; end

        imgID = int32(ann.image_id);
        catID = ann.category_id;

        if isKey(imgID_to_idx, imgID)
            idx = imgID_to_idx(imgID);
            bbox = ann.bbox(:)'; % Ensure it is a 1x4 row vector [x, y, w, h]

            % Assign the bounding box to the correct target class
            if ismember(catID, class1_IDs)
                peopleBoxes{idx} = [peopleBoxes{idx}; bbox];
```

```
            elseif ismember(catID, class2_IDs)
                vehicleBoxes{idx} = [vehicleBoxes{idx}; bbox];
            end
        end
    end

    % Create MATLAB Ground Truth Table
    gtTable = table(filenames, peopleBoxes, vehicleBoxes, ...
        'VariableNames', {'Filename', 'People', 'Vehicles'});

    % Create Datastores
    imds = imageDatastore(gtTable.Filename);
    blds = boxLabelDatastore(gtTable(:, 2:3));

    % Combine them so the image and bounding boxes are passed together
    combinedDatastores{d} = combine(imds, blds);

    fprintf('Created Combined Datastore for %-12s:  %d images\n',
splitNames{d}, numImgs);
end
```

```
Created Combined Datastore for Train       :  10742 images
Created Combined Datastore for Validation  :  1144 images
Created Combined Datastore for Test        :  3749 images
```

```
% Extract individual datastores to easily call them in the training phase
dsTrain = combinedDatastores{1};
dsVal   = combinedDatastores{2};
dsTest  = combinedDatastores{3};
```

### 3.2. Data Augmentation Strategy

To enhance the model's robustness and prevent overfitting, an data augmentation pipeline were implemented. This strategy artificially increases the diversity of the training samples without requiring additional storage. The applied transformations include:

- Random Horizontal Flipping: To account for bidirectional traffic and pedestrian movement.
- Random Scaling (0.9x to 1.1x): To improve the model's ability to detect objects at varying distances from the sensor.

These spatial augmentations are applied simultaneously to the thermal images and their corresponding bounding boxes, ensuring the ground truth remains synchronized with the transformed input. Importantly, color-based augmentations were excluded to preserve the integrity of the thermal intensity signatures.

By using a transform-based approach, the system creates new versions of thermal images directly in memory every time it trains.

```
%% Data Augmentation Strategy

% Apply the custom augmentation function to the Training Datastore
dsTrainAugmented = transform(dsTrain, @aux_augment_data);
```

```
Error using nargin
Function aux_augment_data does not exist.

Error in matlab.io.datastore.TransformedDatastore (line 156)
                    tFuncArgs = nargin(fun);

Error in matlab.io.datastore.internal.buildTransformedDatastore (line 65)
    tds = matlab.io.datastore.TransformedDatastore(datastores, fcn, ...

Error in matlab.io.Datastore/transform (line 350)
            dsnew = matlab.io.datastore.internal.buildTransformedDatastore(varargin{:});
```

```
% Verification: Compare an original sample with its augmented version
reset(dsTrain);
reset(dsTrainAugmented);

% Read the same base index from both
originalData  = read(dsTrain);
augmentedData = read(dsTrainAugmented);

figure('Name', 'Augmentation Verification');
subplot(1, 2, 1);
imgOrig = aux_annotate(originalData{1}, originalData{2}, originalData{3});
imshow(imgOrig);
title('Original Thermal Image', 'FontSize', 12);

subplot(1, 2, 2);
imgAug = aux_annotate(augmentedData{1}, augmentedData{2}, augmentedData{3});
imshow(imgAug);
title('Augmented (Random Flip & Scale)', 'FontSize', 12);
```

# 4. Model Architecture Pipeline

## 4.1. Baseline Selection Strategy

Following the pedagogical strategy for Intelligent Vision Systems, the development process is divided into two phases to ensure a robust pipeline before committing to heavy computational tasks.

- **Phase 1:** High-Speed Baselines: The initial objective is to validate data preprocessing and augmentation settings using "faster" networks. These models have lower depth and smaller memory footprints, allowing for rapid iteration.
- **Phase 2:** High-Accuracy Optimization: After identifying the optimal training hyperparameters, the focus shifts to deeper, more "accurate" networks. These models are better suited for the final detection task but require significantly more prediction time and G

#### 4.1.1. Comparative Architecture Overview

The following table summarizes the specific pre-trained networks selected for testing in this project:

```
%% Comparative Table
NetworkTier = {'Fastest Baseline'; 'Standard Baseline'; 'Accuracy Focused';
'Max Performance'};
Architecture = {'SqueezeNet'; 'GoogLeNet'; 'ResNet-50'; 'Inception-v3'};
Depth = [18; 22; 50; 48];
Size = {'5.2 MB'; '27 MB'; '96 MB'; '89 MB'};
Parameters = {'1.24 M'; '7.0 M'; '25.6 M'; '23.8 M'}; % Added typical
Inception-v3 params for completeness

NetTable = table(NetworkTier, Architecture, Depth, Size, Parameters, ...
    'VariableNames', {'Network_Tier', 'Architecture', 'Depth', 'Size',
'Parameters'});

disp('Deep Learning Network Comparison');
disp(NetTable);
```

### 4.2. Transfer Learning Implementation

Transfer learning is a widely used technique in deep learning because it significantly reduces training time and minimizes the need for large labeled datasets. Instead of training a model from scratch, a pre-trained network, typically trained on large-scale datasets such as ImageNet, is reused. The earlier layers of these networks have already learned to detect general visual features such as edges, textures, and simple shapes, which are useful across many computer vision tasks.

**Layer Replacement:**

The pre-trained network is adapted by replacing its final layers, specifically the Fully Connected layer and the Classification layer. These original layers are designed for the source task (e.g., 1000-class classification in ImageNet) and are not suitable for the new problem.

**Output Adaptation:**

The output layer is modified to match the requirements of the new classification task. In this case, the original 1000 output classes are reduced to 2 classes: *People* and *Vehicles*. The *Vehicles* category aggregates multiple subclasses, including cars, motorcycles, buses, trains, and trucks, simplifying the classification problem.

## 4.3. Training and Monitoring

The model is trained using the `trainNetwork` function. During this phase, two critical performance metrics are continuously monitored to evaluate the learning progress and overall effectiveness of the model.

**Training Loss:**

Training loss measures the error between the predicted outputs and the true labels. The model uses gradient-based optimization techniques to minimize this loss over time. As training progresses, a decreasing loss indicates that the model is improving its ability to correctly classify the input data.

**Training Accuracy:**

Training accuracy represents the percentage of correctly classified samples during training. It is evaluated across epochs (complete passes through the entire dataset) and iterations (individual updates of the model's weights). Monitoring accuracy helps assess how well the model is learning and whether it is converging toward a reliable solution.

ADICIONAR AQUI FINE-TUNING

## 4.4. Model Comparison and Selection

After training multiple models, a detailed comparison is carried out using specific quantitative metrics. These metrics provide a more comprehensive evaluation of model performance, particularly for classification tasks involving multiple categories grouped into broader classes.

### 4.4.1. Evaluation Metrics

**Accuracy:**

Accuracy measures the overall percentage of correctly classified samples. While it provides a general indication of performance, it may not be sufficient on its own, especially if the dataset is imbalanced.

**Precision:**

Precision evaluates how many of the predicted positive instances are actually correct. For example, in the *Vehicles* class, it measures how many images predicted as vehicles truly belong to that category. High precision indicates a low false positive rate.

**Recall:**

Recall measures the ability of the model to correctly identify all relevant instances. In this context, it reflects how well the model detects all actual *People* or *Vehicles* in the dataset. High recall indicates a low false negative rate.

**Confusion Matrix:**

A confusion matrix provides a detailed breakdown of predictions, showing true positives, true negatives, false positives, and false negatives for each class. This allows for a deeper understanding of where the model is making errors (e.g., misclassifying people as vehicles).

**Validation Loss:**

Validation loss measures how well the model performs on unseen data during training. Lower validation loss indicates better generalization and is useful for detecting overfitting when compared with training loss.

#### 4.4.2. Model Selection

The final model is selected based on a combination of these metrics rather than relying on a single value. Preference is given to models that achieve high accuracy along with strong precision and recall across both classes. Additionally, the confusion matrix is analyzed to ensure that misclassifications are minimized and do not disproportionately affect one class.

By first applying a fast baseline (GoogLeNet) and then a high-accuracy model (ResNet-50), the project requirements for rigorous result analysis and generalization are fulfilled. This methodology validates whether the intricate feature extraction of deeper networks is required to successfully isolate people within complex thermal environments containing various vehicle types.

# 5. Model Analysis and Comparison

## 5.1. High-Speed Baselines